Ministry of Education, Culture and Research of the Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automatic

ANDREI ELCISCEV
Group FAF-243

LABORATORY WORK No. 5
THE FILE SYSTEM WITH YOUR OWN HANDS

Checked by:
Reițman Patricia

Chișinău, 2026

Contents

1 Introduction **2**

2 Exploring the Linux File System **2**

2.1 Directory Hierarchy 2

2.2 File Types 2

2.3 Difference Between `/var/log` and `/usr` 3

3 Inodes, Hard Links, and Symbolic Links **4**

3.1 Inodes 4

3.2 Hard Links 4

3.3 Symbolic Links 5

3.4 Sparse Files 6

4 File Permissions and Access Control **7**

4.1 Permission Structure 7

4.2 Numeric Permissions 7

4.3 The `umask` Command 8

4.4 Directory Permissions 8

4.5 Special Permission Bits 9

5 Disk Layout and System Information **10**

5.1 Block Devices and Partitions 10

5.2 Mounted Filesystems and Disk Usage 11

5.3 CPU and Memory Information 11

5.4 Process Information 12

6 Additional Experiments **13**

6.1 File Identification and Magic Numbers 13

6.2 Finding Setuid Files 14

7 Results and Discussion **14**

8 Conclusion **15**

1 Introduction

This laboratory work examines the structure of the Linux filesystem and the mechanisms used to manage files, directories, and access permissions. It covers the directory hierarchy, file types, inodes, hard links, symbolic links, sparse files, and special permission bits. The work also explores disk partitions, mounted filesystems, and system information available through the virtual `/proc` filesystem.

The practical experiments use command-line utilities such as `ls`, `stat`, `find`, `df`, `du`, `lsblk`, `findmnt`, `chmod`, and `umask`. These tools help examine filesystem metadata, modify access permissions, analyze storage usage, and retrieve information about system resources and running processes.

2 Exploring the Linux File System

2.1 Directory Hierarchy

Linux organizes files and directories in a hierarchical structure that begins at the root directory, represented by `/`. Unlike operating systems that use separate drive letters, Linux presents filesystems under a single directory tree.

Important directories include:

- `/etc` – system-wide configuration files.
- `/home` – personal directories belonging to regular users.
- `/bin` – essential user commands; on many modern distributions, this is a symbolic link to `/usr/bin`.
- `/usr` – applications, libraries, documentation, and other shared software resources.
- `/var` – variable data, including logs, caches, and application data.
- `/tmp` – temporary files.
- `/dev` – device files representing devices and certain kernel interfaces.
- `/proc` – a virtual filesystem exposing process and kernel information.

The directory structure can be examined with the following commands:

```
ls /
find /etc -maxdepth 1 -type f
ls -ld /bin /usr/bin /var/log /usr
```

2.2 File Types

Linux supports several file types, which can be identified by examining the first character of the permissions displayed by `ls -l`.

Symbol	Type	Description
`-`	Regular file	Stores text, program code, binary data, or other content.
`d`	Directory	Contains references to files and subdirectories.
`l`	Symbolic link	Contains a path referring to another file or directory.
`c`	Character device	Provides character-oriented device access.
`b`	Block device	Provides block-oriented access to storage devices.
`p`	Named pipe	Enables communication between processes.
`s`	Socket	Supports communication between processes.

For example, `/etc/passwd` is a regular file, `/etc` is a directory, and `/dev/null` is a character device.

The following command displays file types and permissions:

```
ls -l /etc/passwd /etc /dev/null /tmp /bin
file /etc/passwd
stat /etc/passwd
```

2.3 Difference Between `/var/log` and `/usr`

The `/var/log` directory contains system and application log files. These files change as services run and record events, warnings, errors, and other operational information.

The `/usr` directory primarily contains software, libraries, executable programs, and documentation shared by users and applications. Although files in `/usr` can be updated during software installation or system upgrades, this directory is not primarily intended for frequently changing runtime data.

The size of these directories can be inspected with:

```
du -sh /var/log
du -sh /usr
```

The actual sizes depend on the installed software, system configuration, and accumulated logs.

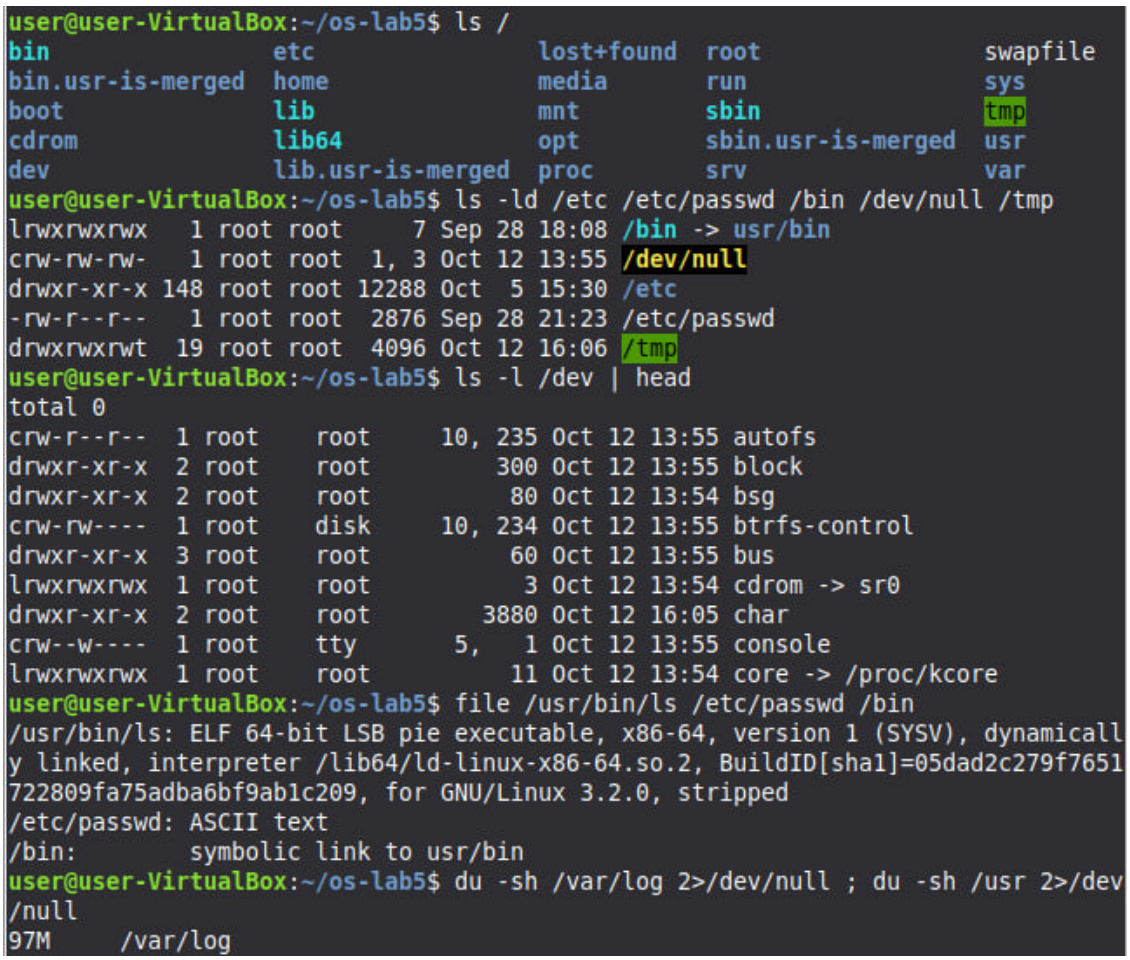


Figure 2.1: Filesystem hierarchy, file types, and directory sizes

3 Inodes, Hard Links, and Symbolic Links

3.1 Inodes

An inode is a filesystem data structure that stores metadata about a file, such as its type, permissions, owner, group, size, timestamps, and references to the file's data blocks.

A filename is stored as a directory entry that associates a name with an inode number. The filename itself is not the inode.

The inode number and metadata of a file can be inspected using:

```
ls -i example.txt
stat example.txt
```

Two different filenames can refer to the same inode when they are hard links to the same file.

3.2 Hard Links

A hard link is another directory entry that refers to the same inode as an existing file. Both names access the same underlying file data.

An example experiment is:

```
echo "Original content" > original.txt
ln original.txt hardlink.txt
ls -li original.txt hardlink.txt
stat original.txt
```

Both files should display the same inode number. The link count increases when the hard link is created.

Changing the contents through either name changes the same underlying file:

```
echo "Updated content" >> hardlink.txt
cat original.txt
```

Deleting one filename does not remove the underlying data while another hard link still refers to the inode. Hard links generally cannot be created for directories by ordinary users and cannot cross filesystem boundaries.

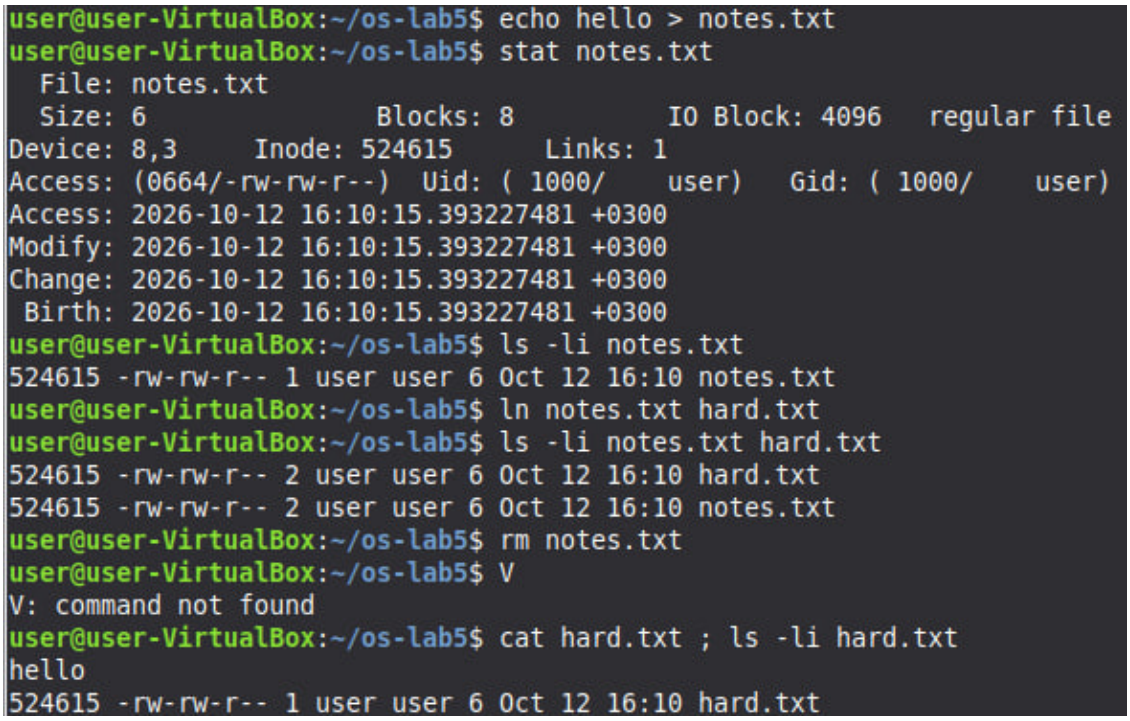


Figure 3.2: Inode numbers and hard-link behavior

3.3 Symbolic Links

A symbolic link is a separate filesystem object that stores a path to another file or directory. Unlike a hard link, it has its own inode and can point to a target on another filesystem.

A symbolic link can be created with:

```
echo "Target content" > target.txt
ln -s target.txt symlink.txt
ls -li target.txt symlink.txt
readlink symlink.txt
cat symlink.txt
```

The symbolic link and its target have different inode numbers. The command `readlink` displays the path stored in the link.

If the target is removed, the symbolic link remains, but it becomes a dangling link:

```
rm target.txt
ls -l symlink.txt
cat symlink.txt
```

The final command reports an error because the target no longer exists.

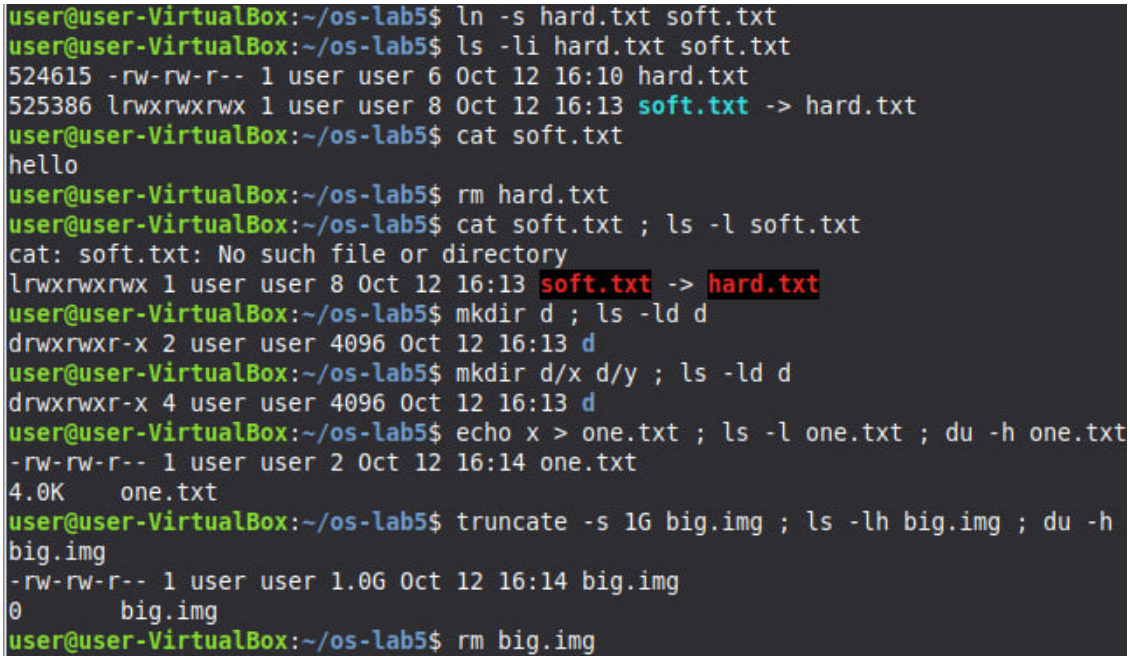


Figure 3.3: Symbolic link behavior before and after removing its target

3.4 Sparse Files

A sparse file is a file whose logical size can be larger than the amount of disk space actually allocated to it. Regions that contain only zeros may be represented as unallocated holes by the filesystem.

A sparse file can be created with:

```
truncate -s 100M sparse.img
ls -lh sparse.img
du -h sparse.img
stat sparse.img
```

The logical size displayed by `ls -lh` may be approximately 100 MB, while `du -h` can report much less allocated storage.

The exact result depends on the filesystem and how the file was created. A sparse file is useful when a program needs a large logical file without immediately allocating storage for every byte.

4 File Permissions and Access Control

4.1 Permission Structure

Linux permissions determine whether a user can read, write, or execute a file. Permissions are divided into three groups:

- Owner – the user who owns the file.
- Group – users associated with the file's group.
- Others – all other users.

Each group can have the following permissions:

- `r` – read, represented by numeric value 4.
- `w` – write, represented by numeric value 2.
- `x` – execute, represented by numeric value 1.

For example, the permission string `-rwxr-xr-` represents a regular file whose owner has read, write, and execute permissions; the group has read and execute permissions; and others have read permission.

4.2 Numeric Permissions

Common numeric permission values include:

Numeric value	Symbolic form	Meaning
600	`rw----`	Only the owner can read and write.
644	`rw-r-r-`	The owner can read and write; others can read.
700	`rwx---`	Only the owner has full access.
755	`rwxr-xr-x`	The owner has full access; others can read and execute.

Permissions can be changed using `chmod`:

```
touch permissions.txt
chmod 600 permissions.txt
ls -l permissions.txt

chmod 644 permissions.txt
ls -l permissions.txt

chmod 755 permissions.txt
ls -l permissions.txt
```

These commands demonstrate how the permissions displayed by `ls -l` change.

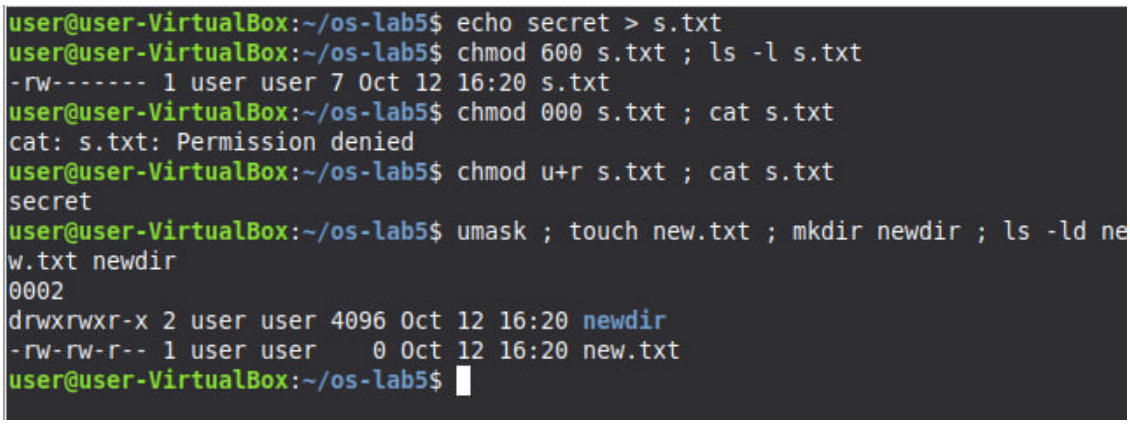


Figure 4.4: File permissions and their modification using chmod

4.3 The `umask` Command

The `umask` command defines which permission bits are removed from the default permissions when new files and directories are created.

The current mask can be displayed using:

```
umask
umask -S
```

A common value is `0022`. For ordinary files, whose initial maximum permissions are typically `666`, this results in permissions of `644`. For directories, whose initial maximum permissions are typically `777`, this results in permissions of `755`.

The calculation is a bitwise removal of permission bits, rather than ordinary decimal subtraction.

The effect can be demonstrated with:

```
umask 0022
touch new_file.txt
mkdir new_directory
stat -c "%a %n" new_file.txt new_directory
```

The expected permissions are usually `644` for the file and `755` for the directory, assuming the usual creation modes and no application-specific changes.

4.4 Directory Permissions

Directory permissions have slightly different meanings from regular-file permissions:

- Read permission allows a user to list directory entries.
- Write permission allows a user to create, remove, or rename entries, subject to other access-control rules.

- Execute permission allows a user to traverse the directory and access entries when the remaining permissions allow it.

For example, a user may be able to access a file inside a directory without being able to list all filenames in that directory, provided the user knows the file's name and has the necessary traversal and file permissions.

A basic experiment can be performed in a test directory:

```
mkdir permission_test
touch permission_test/example.txt
chmod 600 permission_test/example.txt
chmod 700 permission_test
ls -ld permission_test
ls -l permission_test
```

Permission experiments should be performed on files and directories owned by the current user. Changing permissions on system directories can interfere with normal system operation.

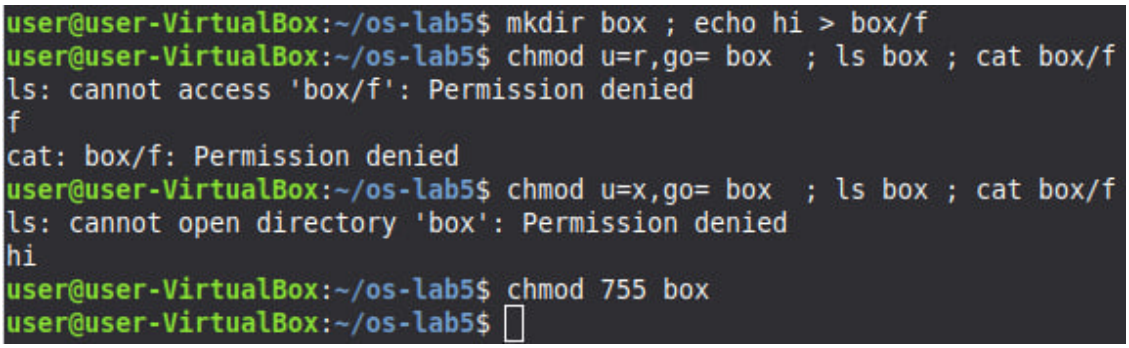


Figure 4.5: Directory read and execute permissions

4.5 Special Permission Bits

Linux supports special permission bits that modify normal execution or directory behavior.

Setuid

The set-user-ID bit allows an executable program to run with the effective user ID of the file's owner, subject to operating-system restrictions. Some system utilities use this mechanism to perform specific privileged operations.

The permissions of the `passwd` utility can be inspected with:

```
ls -l /usr/bin/passwd
stat /usr/bin/passwd
```

On systems where the setuid bit is enabled, the owner's execute position may display `s` instead of `x`. The exact permissions depend on the distribution and system configuration.

Sticky bit

The sticky bit on a shared directory restricts who can remove or rename entries inside it. On a typical Linux system, `/tmp` has the sticky bit enabled so that users cannot ordinarily delete or rename other users' files merely because the directory is writable.

Its permissions can be inspected using:

```
ls -ld /tmp
stat /tmp
```

A permission string such as `drwxrwxrwt` indicates that the sticky bit is set.

Protected files

Files such as `/etc/shadow` contain sensitive account information and are normally restricted to privileged users or authorized system processes.

Their permissions can be inspected using:

```
ls -l /etc/passwd /etc/shadow
stat /etc/passwd /etc/shadow
```

Reading protected files may be denied to an ordinary user. This is expected behavior and should not be bypassed for the purposes of the experiment.

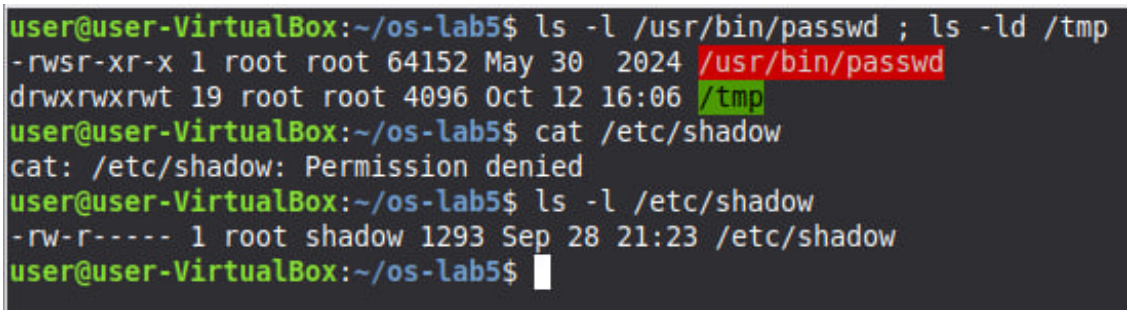


Figure 4.6: Setuid, sticky-bit behavior, and protected system files

5 Disk Layout and System Information

5.1 Block Devices and Partitions

Linux represents storage devices and partitions through device files, commonly located in `/dev`. The exact device names depend on the hardware, virtual machine, and storage configuration.

The following commands display block devices, partitions, filesystems, and mount points:

```
lsblk
lsblk -f
df -h
findmnt
```

For example, an NVMe storage device may appear as `/dev/nvme0n1`, with partitions named `/dev/nvme0n1p1` and `/dev/nvme0n1p2`. A SATA disk may instead use names such as `/dev/sda` and `/dev/sda1`.

These names depend on the storage configuration of the system.

5.2 Mounted Filesystems and Disk Usage

The `df -h` command displays the capacity, used space, available space, and usage percentage of mounted filesystems in a human-readable format.

The `findmnt` command displays information about mounted filesystems, including their mount points and filesystem types.

The `lsblk -f` command provides information about available block devices, filesystems, labels, UUIDs, and mount points.

A filesystem mounted at `/` is the root filesystem. Other filesystems may be mounted at directories such as `/boot`, `/home`, or removable-media mount points.

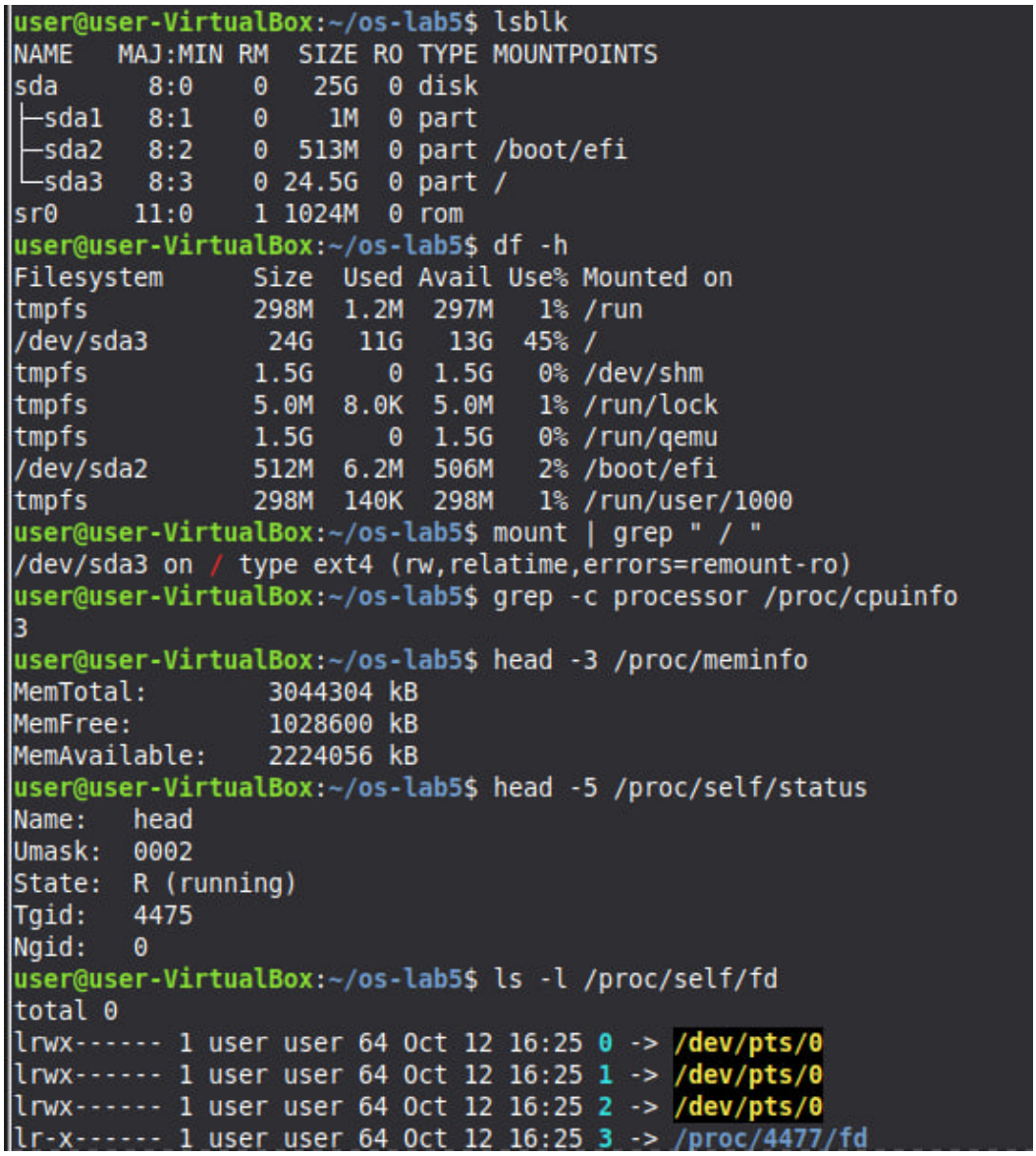


Figure 5.7: Block devices, partitions, filesystem types, and mounted filesystems

5.3 CPU and Memory Information

Linux exposes system information through the virtual `/proc` filesystem. The files in this directory do not behave like ordinary files stored permanently on disk; their contents are

generated by the kernel.

The following commands provide information about the processor and memory:

```
lscpu
cat /proc/cpuinfo
free -h
cat /proc/meminfo
```

The `lscpu` command displays CPU architecture, the number of CPUs visible to the operating system, and other processor details.

The `/proc/cpuinfo` file provides processor-specific information. The `free -h` command summarizes memory usage, while `/proc/meminfo` exposes more detailed memory statistics.

The number of processors visible to Linux may differ from the number of physical CPU cores, especially in virtualized environments.

5.4 Process Information

The `/proc` directory also contains information about running processes. Each process has a directory named after its process ID (PID).

For example, the current shell or a running command can be inspected through its corresponding process directory. The following commands display information about the current shell process:

```
echo $$
cat /proc/$$/status
```

The variable `$$` represents the PID of the current shell. The `status` file contains fields such as the process name, state, process ID, parent process ID, and memory usage.

Some process information is restricted by permissions or system security settings. The available details can also differ depending on the kernel version.

```
user@user-VirtualBox:~/os-lab5$ cat /proc/$$/status
Name:   bash
Umask:  0002
State:  S (sleeping)
Tgid:   2148
Ngid:   0
Pid:    2148
PPid:   2142
TracerPid:      0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
FDSize: 256
Groups: 4 24 27 30 46 100 105 125 1000
NStgid: 2148
NSpid:  2148
NSpgid: 2148
NSsid:  2148
Kthread:        0
VmPeak:    13136 kB
VmSize:    13004 kB
VmLck:         0 kB
VmPin:         0 kB
VmHWM:      6728 kB
VmRSS:      6728 kB
RssAnon:             2108 kB
RssFile:             4620 kB
RssShmem:               0 kB
VmData:     2028 kB
VmStk:       132 kB
VmExe:       956 kB
VmLib:      2608 kB
VmPTE:        64 kB
VmSwap:        0 kB
HugetlbPages:          0 kB
CoreDumping:    0
```

Figure 5.8: CPU, memory, and process information exposed through /proc

6 Additional Experiments

6.1 File Identification and Magic Numbers

A file's type is not always determined by its filename extension. Many file formats contain identifying bytes known as magic numbers, which help utilities recognize the format.

The `file` command examines a file's contents and reports its likely type:

```
file /etc/passwd
file /bin/ls
```

For example, a gzip-compressed file typically begins with the hexadecimal signature `1f 8b`.

A gzip file can be created and inspected with:

```
echo "Example content" > example.txt
```

```
gzip -c example.txt > example.txt.gz
file example.txt.gz
od -An -tx1 -N2 example.txt.gz
```

The `od` command displays the first two bytes in hexadecimal form. For a valid gzip file, these bytes are normally `1f 8b`.

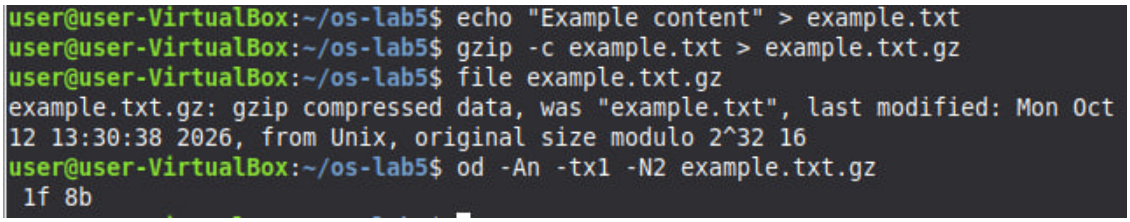


Figure 6.9: Gzip magic number and file identification

6.2 Finding Setuid Files

The `find` utility can search a filesystem for files with particular permission bits. To search common system directories for setuid files, use:

```
find /usr/bin /usr/sbin -type f -perm -4000 2>/dev/null
```

The `-type f` option restricts the search to regular files. The `-perm -4000` expression matches files with the setuid bit set, and `2>/dev/null` suppresses permission-related error messages.

The results depend on the installed software and the system's configuration. Setuid programs should be inspected carefully because they can perform operations using elevated privileges.

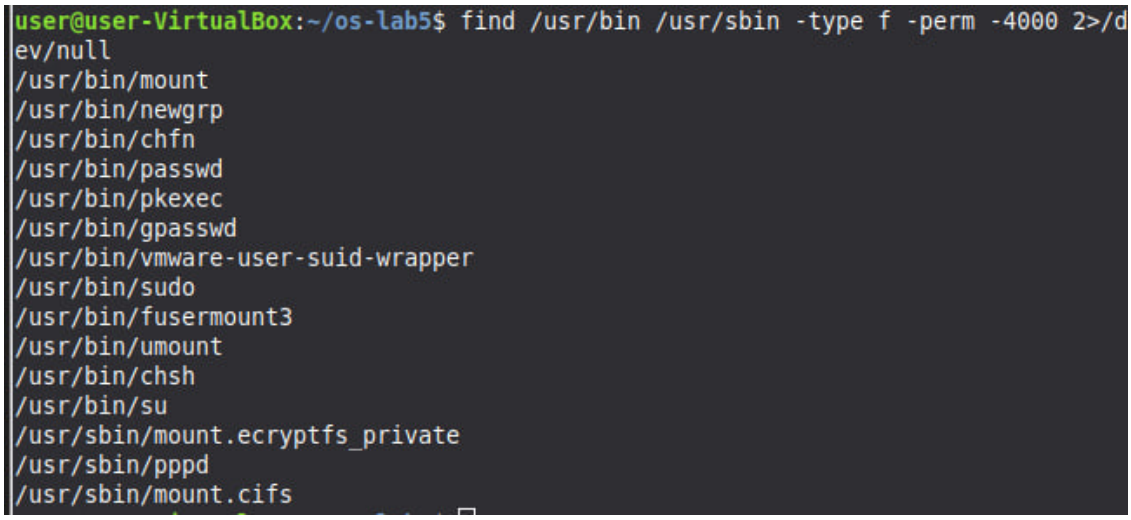


Figure 6.10: Setuid programs discovered using find

7 Results and Discussion

The laboratory work demonstrated the basic organization and behavior of the Linux filesystem.

The directory hierarchy provides a standardized way to organize configuration files, applications, logs, user data, devices, and kernel information. Inodes store file metadata, while directory entries associate filenames with inode numbers. Hard links provide multiple names for the same underlying file, whereas symbolic links refer to paths and can become dangling when their targets are removed.

File permissions control access for owners, groups, and other users. The `umask` influences the permissions of newly created files and directories, while special permission bits provide additional behavior for executable files and shared directories.

The experiments with `lsblk`, `df`, `findmnt`, and `/proc` demonstrated how Linux exposes storage layout, mounted filesystems, processor details, memory statistics, and process information. Finally, file identification and permission searches illustrated how command-line tools can inspect file formats and security-related attributes.

The practical exercises provided a clearer understanding of how Linux manages files and system resources. By examining inode numbers, creating links, modifying permissions, and inspecting system information, the experiments demonstrated how filesystem structure and access control work together. The results also highlighted the importance of command-line utilities for investigating system configuration, identifying files, and locating executables with special permissions.

8 Conclusion

This laboratory work provided practical experience with the Linux filesystem and its principal concepts. The experiments illustrated how files are organized, how inodes and links work, how access permissions are represented and modified, and how special permission bits affect system behavior.

The use of command-line utilities also demonstrated how to examine mounted filesystems, disk usage, hardware information, and running processes. These skills are important for Linux administration, troubleshooting, software development, and understanding the security model of Unix-like operating systems.

The results confirm that the Linux filesystem is more than a directory structure: it is an interface through which users and programs access files, devices, process information, and other operating-system resources.